

# Take Home Assessment (Backend)

## Full Stack Java Software Architect — Fintech, AWS, Microservices

### Objective

**Build a production-ready Wallet & Transfer service that demonstrates the core concerns of a fintech backend: safe money movement, idempotency, data integrity, and resilient API design. We are less interested in feature volume and more in how correctly and cleanly you handle money, concurrency, and failure.**

### Functional Requirements

**Accounts & balances:** Expose an API to create wallet accounts and query a wallet's balance and transaction history. A wallet has a single base currency (e.g., USD).

**Money transfer:** Transfer funds from one wallet to another via a single API call. A transfer must move money atomically — it either fully succeeds or has no effect.

**Idempotency:** The transfer endpoint must be safe to retry. If the client sends the same request twice (same idempotency key), the money must move only once, and the second call must return the original result — not an error and not a double charge.

**Double-entry ledger:** Every transfer must be recorded as balanced ledger entries (a debit and a matching credit). A wallet's balance must be reconcilable from the ledger — the ledger is the source of truth, not a single mutable number.

**Concurrency safety:** Two transfers leaving the same wallet at the same time must never overdraw it. Negative balances are not allowed. Show us where and how the race is prevented.

**Consistent error model:** Return structured, predictable errors (e.g., insufficient funds, wallet not found, validation error, idempotency conflict) with appropriate HTTP status codes.

## Technical Requirements

- Clean separation of layers (API / domain / persistence). We should be able to read the domain logic without wading through framework code.
- A real relational database with **schema migrations** (Flyway or Liquibase) checked into the repo. No “auto-create on startup” as the only mechanism.
- Correct use of database transactions for the money-moving path.
- Tests that prove the hard parts, specifically:
  - an **idempotency test** — the same request twice moves money once;
  - a **concurrency test** — simultaneous transfers cannot overdraw a wallet.
- **Dockerised**: a `docker-compose up` (or equivalent) that brings up the app and its database so we can run it without manual setup.
- API documentation (OpenAPI/Swagger) or a clear, runnable request collection.
- Structured logging on the transfer path.

## Recommended Stack

**Please follow the recommended stack. If you are genuinely more expert in another modern backend stack, you may use it — but be ready to justify the choice in the interview.**

| Area         | Recommended                                   |
|--------------|-----------------------------------------------|
| Language     | Java 17+                                      |
| Framework    | Spring Boot                                   |
| Architecture | Layered or Hexagonal, clear domain boundaries |
| Database     | PostgreSQL                                    |
| Migrations   | Flyway or Liquibase                           |
| API docs     | OpenAPI / Swagger                             |
| Testing      | JUnit 5, Testcontainers, Mockito              |
| Build        | Maven or Gradle                               |
| Run          | Docker + docker-compose                       |

## Stretch Goals (optional)

**Only after the core is solid. These map to the seniority of the role — attempt what you can defend, not everything.**

- Emit a `TransferCompleted` event using the **outbox pattern** (reliable event publishing without a distributed transaction).
- Add a simulated external dependency (e.g., a fraud check) protected by a **timeout, retry, and circuit breaker**.
- Support a second currency with FX conversion on transfer, without breaking existing API consumers.

## What We Evaluate

- **Correctness under stress** — idempotency and concurrency are handled at the right layer (preferably the database), not hopefully in application code.
- **Data integrity** — the ledger balances; the design makes incorrect states hard to reach.
- **Engineering discipline** — meaningful tests, clean structure, readable commits.
- **Production thinking** — migrations, error model, logging, and a clear path to run it.
- **Communication** — a README that explains your decisions and trade-offs.

## Your README Must Include

- 1) How to run the project from a clean checkout.
- 2) A short architecture overview (layers and why).
- 3) Exactly how you guarantee idempotency and prevent overdraw — point to the code.
- 4) One thing you would do differently or add with more time.
- 5) A brief note: how would you add a new NOT NULL column to the transactions table in production with zero downtime? (No code needed — just the approach.)

## Timeline & Submission

**Deadline:** You have 3 days (72 hours) to complete this task, starting from the moment you received this email.

**Version control:** We place high value on your development process. Please provide frequent, descriptive commit messages that reflect your progress and decision-making throughout the 3-day journey.

**Submission:** Reply to this email thread with a link to your public GitHub repository before the deadline.

*If you aren't comfortable with our recommended tech stack, you are welcome to use the languages and frameworks you know best — but the requirements on idempotency, concurrency, and the ledger still apply.*